SkillGraph Technical Architecture & Workflow Specification

Complete End-to-End Request Lifecycle, Pipeline Engines, Docker vs. Cloud Deployment, and Operating Guide (v2.1.0)

> **Verified System State:** All 172 automated test cases pass with 100% green status across Python 3.13, FastAPI 0.162, SQLite, and vanilla HTML5/CSS3 frontend.

1. Executive Summary & Philosophy

SkillGraph is a full-stack career readiness platform. It bridges the gap between candidate resumes/coding profiles and verified engineering roles. It replaces unverified hiring claims with an evidence-aware skill taxonomy across 10 traditional and emerging roles (including *Harness Engineer*, *Forward Deployed Engineer*, and *Agentic AI Engineer*).

The frontend is built strictly with semantic HTML5, pure CSS3 (OKLCH parchment design tokens), and vanilla modern JavaScript (ES2022+), completely eliminating external heavy build frameworks (no React, Next.js, or npm bundlers). The backend runs on FastAPI with SQLite database persistence and asynchronous external profiling integrations.

2. Complete End-to-End Request & Workflow Lifecycle

The diagram and sequence below describe what occurs under the hood when a user interacts with the application:

Stage 1: Client Request Initiation (Browser)

The user interacts with the vanilla UI (e.g., clicks "Analyze Resume", "Check Coding Profiles", or "Generate Roadmap"). The browser executes an asynchronous `fetch()` request configured in `frontend/js/api.js` sending JSON or multipart/form-data payloads to `http://127.0.0.1:8000/api/...`.

Stage 2: ASGI Middleware, Security & CORS Filtering

The FastAPI ASGI application receives the raw TCP packet on Uvicorn port 8000. It passes through:

- **CORS Middleware:** Inspects the `Origin` header against authorized local origins (e.g., `http://127.0.0.1:8000`, `http://localhost:3000`), blocking unauthorized origins.
- **Payload Size Guard:** Checks file stream length, rejecting uploads exceeding 5 MB with `HTTP 413`.
- **JWT Authentication Guard:** Validates Bearer tokens for protected endpoints (upload/save) or grants unauthenticated access for guest critique and exploration.

Stage 3: Pydantic Schema Validation & Sanitization

Incoming request bodies are deserialized and strictly validated against Pydantic v2 models (e.g., `ResumeCritiqueRequest`, `CodingProfileRequest`). Malformed payloads, empty strings, or invalid handles trigger immediate `HTTP 422` validation errors with field-level diagnostic messages.

Stage 4: Domain Service Pipeline Execution

The validated payload is dispatched to the corresponding service module:

- **Resume Parsing & Magic Byte Audit:** Checks byte headers (`%PDF-` or `PK`) to prevent extension spoofing. PyMuPDF or python-docx extracts full text.
- **Skill Extraction Engine:** Scans extracted text against the canonical skill taxonomy using boundary-aware regex tokens, matching aliases, multi-word skills, and case variations.
- **Resume Quality & Advisor Engine:** Computes a 0–100 composite score across 4 dimensions: (1) Metric

Quantification via Google XYZ patterns, (2) Action Verb strength, (3) Keyword Density, and (4) Fluff Word / Cliché penalties.
- **Coding Profile Harvester:** Dispatches non-blocking async `httpx` queries to the GitHub REST API (repos, stargazers, language mix) and LeetCode GraphQL (Easy/Medium/Hard problem distribution). Computes the *DSA Interview Readiness Index*.
- **Job Market Analytics Aggregator:** Analyzes cross-role skill frequencies and computes co-occurrence pairs across roles.
- **Roadmap Synthesis Engine:** Resolves topological skill dependencies and constructs a personalized 6-week progressive milestone schedule.

Stage 5: SQLite Database Transaction & Persistence

For operations requiring persistence (resume storage, user accounts, custom roadmaps, task completion tracking), SQLAlchemy executes parameterized SQL queries within ACID transactions in the SQLite database (`skillgraph.db`). Cascading deletions guarantee referential integrity.

Stage 6: Serialization & Response Dispatch

Results are serialized into Pydantic response models, converted to JSON, and returned with HTTP 200/201 status codes.

Stage 7: Vanilla JS UI Hydration & Dynamic Rendering

The client's `api.js` promise resolves. Dedicated page scripts (e.g., `resume-review.js`, `coding.js`, `analytics.js`) construct semantic HTML elements, calculate SVG progress meters, and render interactive elements without triggering page reloads.

3. Docker vs. Cloud Deployment: Why Docker Alone Does Not Produce a Public Link

> **Important Clarification on Docker:**
> A common misconception is that building or running a Docker container automatically produces a public website link (like `*.lovable.app` or `*.vercel.app`).

Here is the exact difference:

Concept	What It Actually Does	Where It Runs / Accessible URL
Local Docker	Packages application code, Python runtime, and libraries into a self-contained image on your own computer.	Runs on your local computer CPU. Accessible only to you at `http://localhost:8000`.
Lovable / Vercel Hosting	A commercial cloud platform that runs massive servers, owns root domains, configures DNS, and assigns public subdomains.	Runs in a cloud data center. Accessible to the entire world at `https://your-app.lovable.app`.
Cloud Container Hosting (Render / Railway)	Takes your repository Dockerfile, builds the container in the cloud, attaches an SSL certificate and public URL.	Runs in the cloud. Accessible to the entire world at `https://skillgraph.onrender.com`.

4. How to Deploy SkillGraph to Get a Free Public Link in 2 Minutes

Since the repository is already synchronized with GitHub at `https://github.com/JasmineChander06/SkillGraph`, you can deploy it to Render for free:

1. Sign in to **https://dashboard.render.com** with your GitHub account.

2. Click **New +** and select **Web Service**.
3. Connect the **SkillGraph** repository.
4. Render automatically detects the `Dockerfile`.
5. Set Environment Variables:
   - `PORT` = `8000`
   - `SECRET_KEY` = `production_secret_key_skillgraph_2026`
6. (Optional) Add a Persistent Disk mounted at `/app/data` for permanent SQLite persistence.
7. Click **Create Web Service**. Render builds the container and assigns you a live public HTTPS URL!

5. How to Run and Use SkillGraph Locally Right Now

Choose whichever method you prefer:

Method A: One-Click Windows Launcher (Easiest)

Simply navigate to the project directory and double-click `run.bat`. It starts the server on port 8000.

Method B: Direct PowerShell Commands

```
Set-Location c:\Users\jasmi\OneDrive\Desktop\SkillGraph\backend
.\.venv\Scripts\python.exe -m uvicorn app.main:app --host 127.0.0.1 --port 8000
```

Method C: Docker Commands (Local Container)

```
docker build -t skillgraph .
docker run -d -p 8000:8000 --name skillgraph_app skillgraph
```

6. Interactive Feature URLs & User Journey

Feature	Local Live URL	Interactive Action
Home / Landing	`http://127.0.0.1:8000/frontend/index.html`	Overview, architecture highlights, quick navigation.
Dashboard	`http://127.0.0.1:8000/frontend/dashboard.html`	Upload resume, view readiness radar chart and skill breakdowns.
Resume Reviewer	`http://127.0.0.1:8000/frontend/resume-review.html`	Audit text, view 0-100 rubric, items to add/remove, XYZ bullet rewrites.
Coding Profiles	`http://127.0.0.1:8000/frontend/coding.html`	Fetch GitHub activity + LeetCode DSA breakdown and readiness index.
Market Analytics	`http://127.0.0.1:8000/frontend/analytics.html`	Explore cross-role skill demand and high-signal pairings.
Role Explorer	`http://127.0.0.1:8000/frontend/roles.html`	Compare requirements (e.g. Harness Engineer vs Backend Engineer).
Roadmap	`http://127.0.0.1:8000/frontend/roadmap.html`	Track weekly milestones, mark topics completed.
API Swagger Docs	`http://127.0.0.1:8000/docs`	Test interactive endpoints and view OpenAPI 3.1 schema.

7. Automated Test Suite Verification (172 / 172 Passed)

Test File	Count	Scope Verified	Status

`test_resume_advisor.py`	9	Quantification, verbs, fluff elimination, Google XYZ rewrites	**PASSED**
`test_coding_profiles.py`	5	GitHub repos, LeetCode Easy/Med/Hard, DSA Readiness	**PASSED**
`test_job_market_analytics.py`	3	Role demand, co-occurrences, empirical benchmarks	**PASSED**
`test_frontend_integration.py`	2	21 static assets & complete 15-step lifecycle	**PASSED**
`test_resume.py`	10	PDF/DOCX magic byte validation, corrupt file handling	**PASSED**
`test_skills.py`	36	Taxonomy boundary matching, aliases, coverage calculations	**PASSED**
`test_role_discovery.py`	31	Traditional & emerging roles catalog, comparisons	**PASSED**
`test_job_description.py`	17	JD parsing, requirements segmentation	**PASSED**
`test_roadmap.py`	16	6-week DAG prerequisite scheduling, progress status	**PASSED**
`test_auth.py`	16	Password hashing, JWT issuance and verification	**PASSED**
`test_workflow.py`	11	Multi-step workflows, IDOR isolation	**PASSED**
`test_database.py`	11	Table creation, foreign keys, cascades, rollback safety	**PASSED**
`test_health.py`	5	Health status, CORS whitelist & rejection	**PASSED**
Total Suite	**172**	**Complete Full-Stack Application Verification**	**172 / 172 PASSED**